# Project One

Numerical Analysis

December 31, 1979

Logan Reed

# 1 Problems

**Problem 1.** See the supplied code and figures for the interpolating polynomials. The "Ordering Error" measures the difference between the two orders one could compute the polynomial.

**Problem 2.** Yes, the choice of points made a difference which can be seen with the relative error measure. The chebischev points seem to give better results, which should be expected as it did the same in the one dimensional version of the function.

**Problem 3.** Consider the real numbers $x_i$, $D^\alpha f_i$ for $i = 0, \ldots, m$ and $|\alpha| = 0, 1, \ldots, n_i - 1$ with

$$x_0 < x_1 < \ldots < x_m.$$

The *Multidimensional Hermite interpolation Problem* consists of determining a polynomial $P \in \Pi_n$, where

$$n + 1 := \sum_{i=0}^{m} n_i,$$

and which satisfies the following interpolation conditions:

$$D^\alpha P(x_i) = D^\alpha f_i, \quad i = 0, 1, \ldots, m, \quad |\alpha| = 0, 1, \ldots, n_i - 1.$$

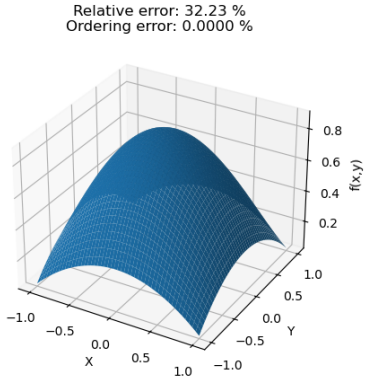
The wanted theory is

**Conjecture 1.** *For arbitrary numbers $x_0 < x_1 < \ldots < x_m$ and $D^\alpha f_i$ for $i = 0, \ldots, m$ and $|\alpha| = 0, 1, \ldots, n_i - 1$ there exists a polynomial which solves the Multidimensional Hermite interpolation problem given by*

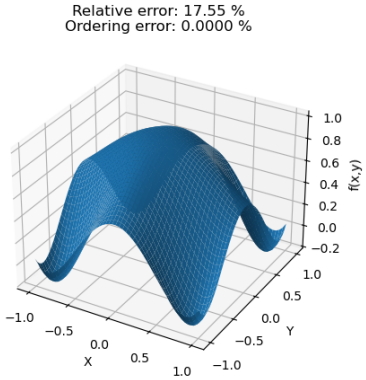
$$P \in \Pi_n, \quad n + 1 := \sum_{i=0}^{m} n_i.$$

This is a slight modification of theorem (2.1.5.2) in the textbook, and the proof should be similar to their respective proof. I did not include any uniqueness statement since we need to restrict the collection of $x_i$'s to have specific properties, but one could add this restriction. The code implements a regular grid which, I believe, guarantees uniqueness.

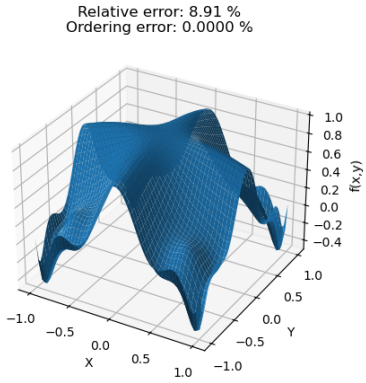
**Problem 4.** Yes, the choice of points made a difference which can be seen with the relative error measure. It seems that the Chebischev points help to dampen the oscillations near the boundary. That being said, there seems to be a small bug in my implementation which could be influencing the error measure. Obviously we want the ordering error to be 0, and the Hermite Interpolation gets a fairly large (decreasing in $N$) ordering error. For project one I decided to implement Lagrange Interpolation recursively just for fun, and I believe a significant part of the error is due to the number of floating point operations python has to do. With that in mind, the difference between the Hermite and Lagrange Interpolation's are fairly similar.



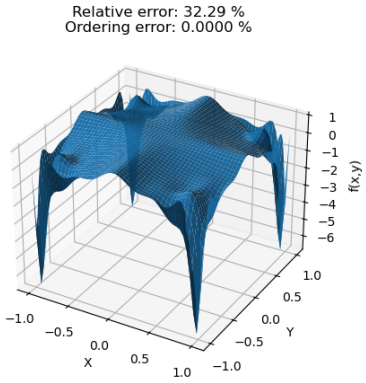

(a) N=4




(b) N=6

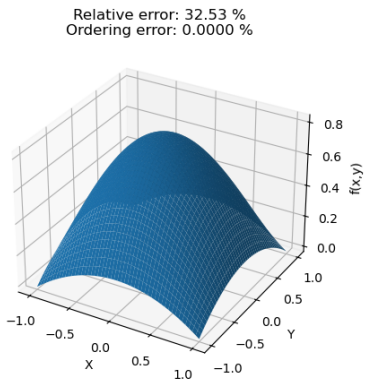



(c) N=9

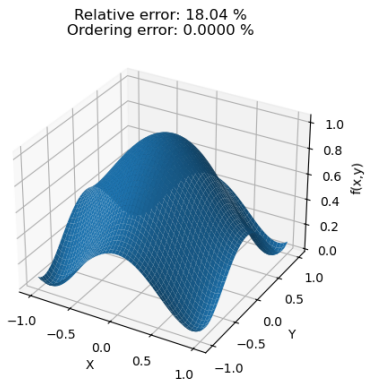



(d) N=13

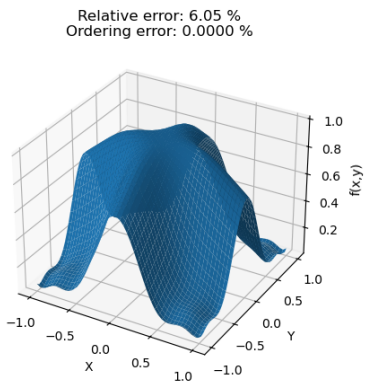
Figure 1: Equidistant points, Lagrange Interpolation, $f(x) = \frac{1}{1+25x^2y^2}$ with $N$ points



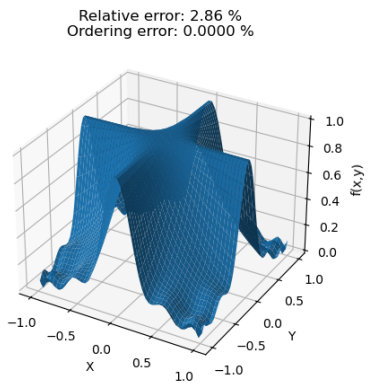

(a) N=4




(b) N=6

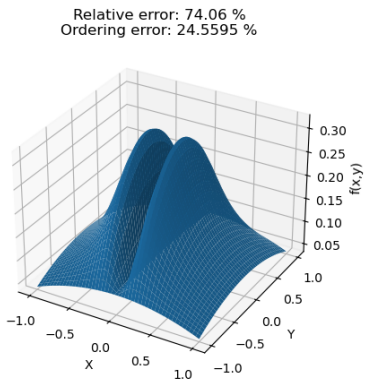



(c) N=10

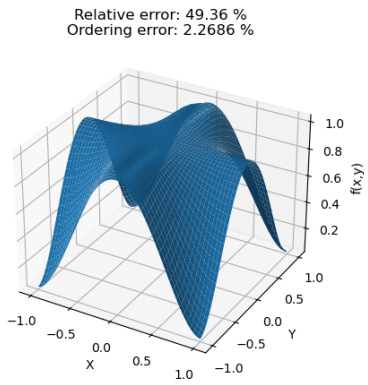



(d) N=13

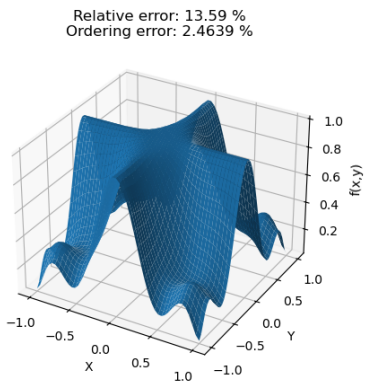
Figure 2: Chebishev points, Lagrange Interpolation, $f(x) = \frac{1}{1+25x^2y^2}$ with $N$ points



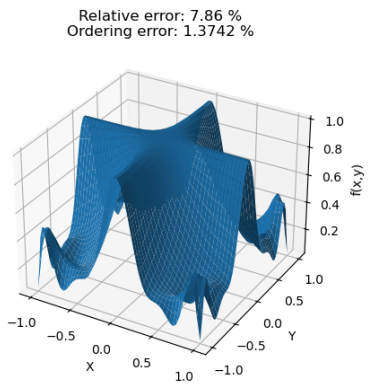

(a) N=5




(b) N=7

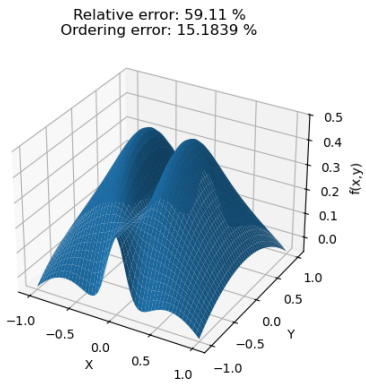



(c) N=11

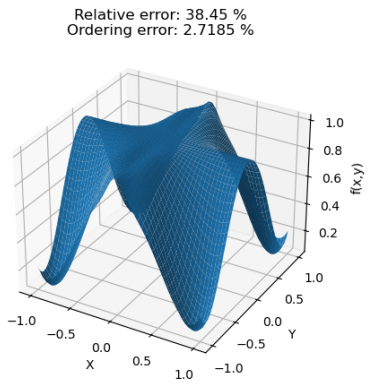



(d) N=15

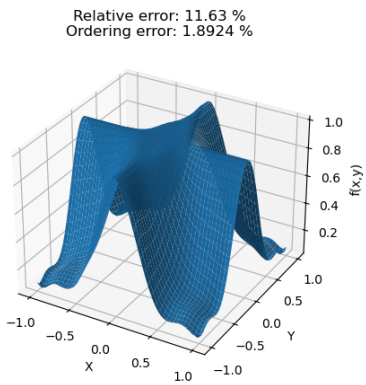
Figure 3: Equidistant points, Hermite Interpolation, $f(x) = \frac{1}{1+25x^2y^2}$ with $N$ points



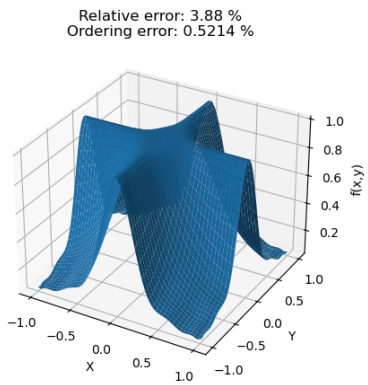

(a) N=5




(b) N=7




(c) N=11




(d) N=15

Figure 4: Chebishev points, Hermite Interpolation, $f(x) = \frac{1}{1+25x^2y^2}$ with $N$ points

# A Appendix

```python
import numpy as np
from itertools import combinations
import matplotlib.pyplot as plt


def D( i, k, Xn, Yn ):
    if i+k>=len(Xn):
        return 0
    elif k == 0:
        return Yn[i]
    else:
        return (D(i+1, k-1, Xn, Yn)-D(i, k-1, Xn, Yn))/(Xn[i+k]-Xn[i])

def Dh( j, k, Zn, Yn, Ypn ):
    if j+k>=len(Zn):
        return 0
    elif k == 0:
        return Yn[int(j/2)]
    elif k == 1 and j%2 == 0:
        return Ypn[int(j/2)]
    else:
        return (Dh(j+1, k-1, Zn, Yn, Ypn)-Dh(j, k-1, Zn, Yn, Ypn))/(Zn[j+k]-Zn[j])

def Lagrange( x, Xn, Yn ):
    n = len(Xn)
    try:
        Ninter = len(x)
    except:
        Ninter = 1
        x = np.array([x,])

    Lp = D(0, 0, Xn, Yn)*np.ones(Ninter)
    for l in range(Ninter):
        for k in range(1, n):
            Lp[l] += D(0, k, Xn, Yn)*np.prod(x[l]-Xn[:k])
    return Lp

def Hermite( x, Xn, Yn, Ypn ):
    n = len(Xn)
    Zn = np.zeros( 2*len(Xn) )
    Zn[::2] = Xn
    Zn[1::2] = Xn
    try:
        Ninter = len(x)
    except:
        Ninter = 1
        x = np.array([x,])

    Hp = Dh(0, 0, Xn, Yn, Ypn)*np.ones(Ninter)
    for l in range(Ninter):
        for k in range(1, 2*n+2):
            Hp[l] += Dh(0, k, Zn, Yn, Ypn)*np.prod(x[l]-Zn[:k])
    return Hp
```

```python

def function2(x,y=1):
    return 1 / (1 + 25 * np.power(x,2) * np.power(y,2))

def dfunction2x(x,y=1):
    return (-50 * x * np.power(y,2)) / np.power((1 + 25 * (np.power(x,2) * (np.power(y,2)))),2)

def dfunction2y(x,y):
    return (-50 * y * np.power(x,2)) / np.power((1 + 25 * (np.power(x,2) * (np.power(y,2)))),2)

def chebyshev_points(a, b, n):
    i = np.array(range(n))
    x = np.cos((2*i+1)*np.pi/(2*(n)))
    return 0.5*(b-a)*x+0.5*(b+a)

def intpol(x,x2,N,M,hermite=False,chebishev=False):
    Ndat = N
    Ninter = M
    xmin = -1
    xmax = 1
    f = np.zeros([Ninter,Ninter])

    if not chebishev:
        Xn = np.linspace( xmin, xmax, Ndat )
        X2n = np.linspace( xmin, xmax, Ndat )
    else:
        Xn = np.sort(chebyshev_points(xmin,xmax,Ndat))
        X2n = np.sort(chebyshev_points(xmin,xmax,Ndat))
    i = 0
    g = np.zeros([Ndat,Ninter])
    for xx in X2n:
        Yn = function2(Xn, xx)
        Ypn = dfunction2x(Xn, xx)

        if hermite:
            y = Hermite( x, Xn, Yn, Ypn )
        else:
            y = Lagrange( x, Xn, Yn )


        g[i,:] = y

        i = i + 1

    for goal in range(len(x2)):
        Yn = g[:,goal]
        Ypn = dfunction2y(x[goal], X2n)

        if hermite:
            y = Hermite( x, X2n, Yn, Ypn )
        else:
            y = Lagrange( x, X2n, Yn )
        f[:,goal] = y
    return f

```

```python



def start(Ndat, Ninter, hermite=False, chebishev=False):
    x = np.linspace(-1,1,Ninter)
    y = np.linspace(-1,1,Ninter)
    f = np.zeros([Ninter,Ninter])
    for i in range(Ninter):
        for j in range(Ninter):
            f[i,j] = function2(x[i], y[j])
    fig = plt.figure()
    ax = fig.add_subplot(111, projection='3d')
    X, Y = np.meshgrid(x, y)
    zs = intpol(x,y,Ndat,Ninter, hermite,chebishev)
    zs_rev_order = intpol(y,x,Ndat,Ninter, hermite,chebishev).T
    error = np.sum((np.abs(f - zs))) / np.sum(np.abs(f)) * 100
    error2 = np.sum((np.abs(zs - zs_rev_order))) / np.sum(np.abs(zs)) * 100
    plt.title(f'Relative error: {error:0.2f} %\nOrdering error: {error2:0.4f} %')
    Z = zs.reshape(X.shape)
    ax.plot_surface(X, Y, Z)

    ax.set_xlabel('X')
    ax.set_ylabel('Y')
    ax.set_zlabel('f(x,y)')
    if hermite:
        if chebishev:
            plt.savefig(f'output/ch_herm_{Ninter}pt_{2*Ndat+1}data.png')
        else:
            plt.savefig(f'output/eq_herm_{Ninter}pt_{2*Ndat+1}data.png')
    else:
        if chebishev:
            plt.savefig(f'output/ch_{Ninter}pt_{Ndat}data.png')
        else:
            plt.savefig(f'output/eq_{Ninter}pt_{Ndat}data.png')

if __name__ == "__main__":
    start(9,50 ,hermite=True,chebishev=True)

```